School of Computing and Information Systems

COMP30023: Computer Systems

Tutorial Week 3

Introduction to Processes and Threads

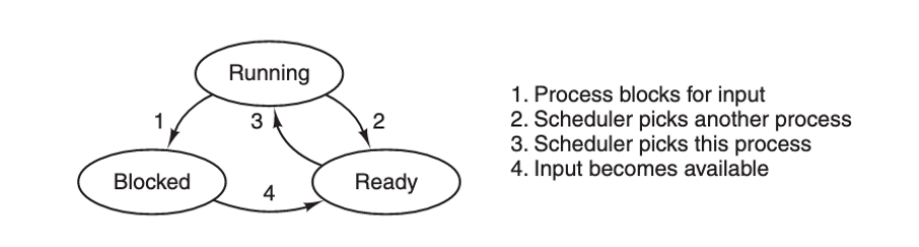
Q1: What is the difference between kernel and user mode? Explain how having two distinct modes aids in designing an operating system?

A1: Most modern CPUs provide two modes of execution: kernel mode and user mode. The CPU can execute every instruction in its instruction set and use every feature of the hardware when executing in kernel mode. However, it can execute only a subset of instructions and use only a subset of features when executing in user mode. Having two modes allows user programs to be restricted to executing (non-privileged) instructions that do not interfere with managing the hardware resources, the operating system, or other processes.

Q2: What is the purpose of a system call in an operating system?

A2: System calls provide an interface for user programs to interact with the operating system. Through system calls, user programs can request the operating system to execute privileged instructions on their behalf. For example, a user program might invoke a system call to request the operating system to read data from a file.

Q3: Three process states are shown in the state transition figure below. In theory, with three states, there could be six transitions, two out of each state. However, only four transitions are shown. Identify the missing transitions and explain whether they can occur and under what circumstances they might occur.



A3: Transition 1: Blocked to Running
A process typically cannot transition directly from Blocked to Running. A Blocked process must first transition to Ready before transitioning to Running. This is because the OS scheduler is responsible for transitioning processes to the Running state and operates only over Ready processes. Even if the CPU is idle when the event blocking a process occurs, the process must still be placed in the Ready state for it to be considered by the scheduler for execution.

Additional transition 2: Ready to Blocked
This transition can never occur. A ready process is sitting in a queue waiting for CPU time; it cannot execute instructions and, hence, cannot initiate any I/O or any other event that might block it. Only a running process can block.

Q4: Suppose you are writing a program that processes a very large, static array of integers. The program raises each element in the array to the power of n, where n is provided as input. You can assume the entire array is loaded into memory before processing begins. To improve performance, you decide to use multithreading, with each thread processing an independent section of the array. However, your computer has only a single processor with a single core. Explain why this multithreaded implementation will not provide any performance benefits compared to a single-threaded version.

A4: If only one processor is available, then a multithreaded approach to the application described will not improve performance. Although each thread processes a subset of the array's elements, processing all the array elements will still occur sequentially as the threads take turns using the only available processor. Compared to the single-threaded version of the application, the only difference will be the order in which the array elements are processed. This will depend on how the threads are scheduled on the processor. Still, only one element of the array will be processed at a time. Multithreading will introduce additional overhead that will cause the application's performance to worsen. Such overhead comes from the operating system's work when switching between threads.

In general, a multithreaded approach to a CPU-bound application such as the one described will only yield performance benefits in a multi-processor setting, where work can be parallelised by means of threads that can potentially run simultaneously on multiple processors.

Q5: Consider the following code segment:

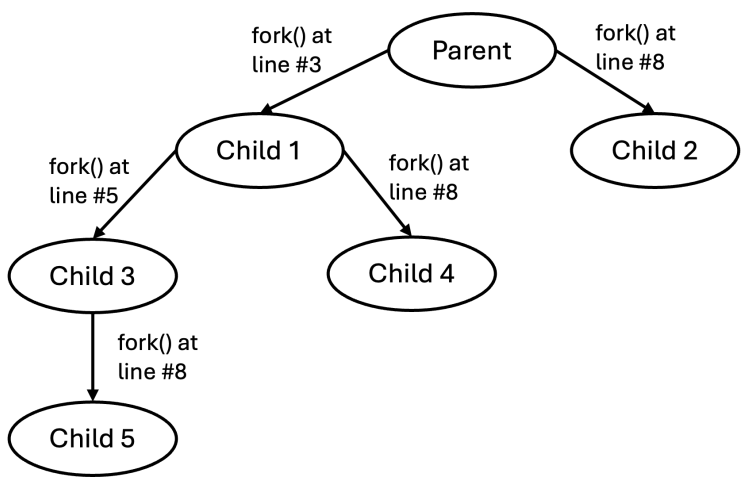
```
1 pid_t pid;
2
3 pid = fork();
4 if (pid == 0) { /* child process */
5   fork();
6   thread_create( . . .);
7 }
8 fork();
```

a. How many processes are created?

b. How many threads are created?

A5:

a. 5 new processes are created:



b. 2 threads are created. Child 1 in the above diagram creates 1 thread, and Child 3 creates 1 thread.